

UNIVERSIDAD TECNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computacion y Diseño Digital

Carrera de Ingeniería de Software

Guía de la Tercera Entrega del Proyecto Fin de Curso (PFC)

Aplicaciones Web — Quinto nivel

Periodo Académico Presencial 2026-2027

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

`gguerrero@uteq.edu.ec`

Etiqueta objetivo de esta entrega: v0.9.0-rc

Fecha de cierre: viernes 24 de julio de 2026 (semana 13)

Precede a: Entrega Final (v1.0.0, 17 de agosto de 2026)

Continúa desde: Entrega 1B (v0.7.0, semana del 14 de junio)
y Entrega 1A (v0.3.0, semana del 4 de junio)

Quevedo — Los Rios — Ecuador

2026

1. Naturaleza y ubicación temporal de la Tercera Entrega

La Tercera Entrega del PFC ocupa el hito intermedio entre la Entrega 1B (v0.7.0, semana del 14 de junio de 2026) y la Entrega Final (v1.0.0, semana del 17 de agosto de 2026, semana 17 del calendario académico). Cierra el 24 de julio de 2026. Su propósito no es cerrar nuevas funcionalidades sino consolidar y verificar, con evidencia empírica reproducible, todo lo entregado hasta la Entrega 1B, elevando el proyecto al estado *release candidate* (v0.9.0-rc) exigido por las normas de reproducibilidad y de reporte de las revistas científicas de alto impacto en ingeniería de software.

El PFC no comienza en la Entrega 1B. Su cimiento formal es el *corpus de ingeniería de requisitos* producido durante la Entrega 1A (semana del 4 de junio) y en el ciclo previo 2025 PAS, que documenta el problema real que el sistema resuelve, los actores del dominio, los objetivos de negocio, los requisitos funcionales y no funcionales, los casos de uso y las historias de usuario. La Tercera Entrega debe conservar la trazabilidad hacia ese corpus y hacer explícita la evolución que los requisitos han tenido desde su primera especificación hasta la versión estable candidata, siguiendo la disciplina de gestión de requisitos formalizada por la especificación internacional *ISO/IEC/IEEE 29148:2018* [11, 20, 29].

Mientras que la Entrega 1B verifica lo que el sistema *hace* (módulo de autenticación *stateless* con JWT sobre *cookie HttpOnly*¹, CRUD sobre entidades del dominio con Spring Data JPA, migraciones Flyway² sobre PostgreSQL 16, *cache Redis*³, cabeceras de seguridad y contenedores orquestados con Docker Compose), la Tercera Entrega verifica *cómo* lo hace, *con qué garantía*, *con qué evidencia empírica cuantitativa* y *con qué grado de reproducibilidad por un tercero sin conocimiento previo del proyecto*. Esta distinción es la que separa un producto software de un artefacto de investigación reutilizable, reproducible y verificable según las normas contemporáneas de gestión de datos y de artefactos de investigación [30, 1, 22].

En consecuencia, la Tercera Entrega solicita al equipo cuatro transformaciones sobre el estado v0.7.0 que ya posee. Cada transformación se ilustra con un ejemplo concreto que aclara cómo se ejecuta en la práctica.

1. **Aplicación íntegra de las observaciones de las Entregas 1A y 1B:** cada observación del docente en los informes de retroalimentación recibidos debe estar resuelta y su resolución debe ser trazable en el historial de *commits*⁴ del repositorio, con referencia explícita a la observación original.

¹**Cookie HttpOnly:** atributo de las *cookies* HTTP que impide su lectura desde código JavaScript del navegador; protege el JWT contra ataques de tipo *Cross-Site Scripting* (XSS).

²**Flyway:** herramienta de control de versiones del esquema de base de datos que ejecuta *scripts* SQL numerados de forma incremental y garantiza que todos los entornos de despliegue (desarrollo, prueba, producción) tengan el mismo esquema.

³**Cache Redis:** almacén en memoria de tipo clave-valor usado para guardar respuestas frecuentes y evitar consultas repetidas a la base de datos.

⁴**Commit:** unidad atómica de cambio en Git; representa un conjunto de modificaciones registradas con autor, fecha y mensaje descriptivo.

Ejemplo — Aplicación de observaciones

Supóngase que en la Entrega 1B el docente comentó: “*el JWT debería incluir el claim **aud** para restringir el público objetivo del token*”. En la bitácora `docs/observaciones/OBSERVACIONES.md` se registra:

Codigo	Fuente	Criterio	Observacion	Decision	Commit
-----	-----	-----	-----	-----	-----
OBS-04	Entrega 1B	Auth JWT	Anadir claim aud al JWT	Aplicada en JwtSvc	a1b2c3d

El mensaje del *commit* correspondiente es:

```
feat(auth): anade claim aud al JWT firmado (OBS-04)
```

2. **Consolidación de la ingeniería de requisitos:** la especificación de requisitos producida en la Entrega 1A se actualiza, se valida y se hace trazable de extremo a extremo hacia el código, las pruebas y la evidencia empírica del sistema, siguiendo el marco de *ISO/IEC/IEEE 29148:2018* y la guía de calidad de requisitos del *INCOSE Requirements Working Group* [11, 10, 20].

Ejemplo — Consolidación de un requisito

Un requisito funcional típico de un sistema de gestión de pedidos podría redactarse así (patrón *[condición] [sujeto] shall [acción] [objeto] [restricción]* de *ISO/IEC/IEEE 29148*):

Id:	REQ-F-014
Tipo:	Funcional
Prioridad:	Must (MoSCoW)
Enunciado:	Al recibir una solicitud de anulacion, el sistema debera cambiar el estado del pedido a ANULADO en un plazo maximo de 2 segundos.
Rationale:	El cliente pierde la confianza si la anulacion tarda mas de 2s (evidencia: entrevista E-03).
Verificacion:	Test JUnit integrationTest_anularPedido_p95_2000ms
Trazabilidad:	-> Historia HU-08 -> Caso de uso CU-05 -> -> Endpoint POST /api/pedidos/{id}/anular -> -> Test PedidoTest.anular_p95 -> Evidencia perf/run-03.json
Estado:	verificado

3. **Consolidación de la reproducibilidad automática:** el proyecto debe poder ser reconstruido de forma idéntica por un tercero desde una clonación limpia, con contenedores anclados por *digest*⁵ `sha256`, semillas deterministas fijadas y *scripts* de un solo comando para levantar, ejecutar pruebas, generar métricas y apagar el sistema [30, 1].

⁵**Digest sha256:** huella criptográfica única de 256 bits que identifica una imagen Docker por su contenido exacto, evitando derivas silenciosas cuando la imagen del registro se modifica pero conserva su etiqueta.

Ejemplo — Reproducibilidad automática en un solo comando

Un revisor externo clona el repositorio y ejecuta:

```
$ git clone https://github.com/<equipo>/<repo>.git
$ cd <repo>
$ git checkout v0.9.0-rc
$ cp .env.example .env
$ make up
```

En menos de dos minutos el sistema completo está operativo. En el `docker-compose.yml` las imágenes van fijadas por *digest*, no por etiqueta variable:

```
services:
  postgres:
    image: postgres@sha256:2f7e3f7...
  redis:
    image: redis@sha256:5c1c8b9...
```

4. **Consolidación de la evidencia empírica cuantitativa y de la trazabilidad:** cada afirmación no trivial del sistema (rendimiento, seguridad, usabilidad, fiabilidad, cobertura) debe estar respaldada por un archivo de mediciones crudo, un *script* de análisis y un reporte agregado con estadística descriptiva e inferencial mínima, y cada requisito funcional y no funcional debe estar trazado hasta su código, su prueba automatizada y su evidencia empírica de cumplimiento [16, 4, 2, 23, 22].

Ejemplo — Evidencia empírica y trazabilidad

La afirmación “el endpoint de listado de productos responde con $p95 \leq 200ms$ con caché caliente” se sustenta con tres archivos versionados:

```
docs/mediciones/perf/k6-run1.json # datos crudos, corrida 1
docs/mediciones/perf/k6-run2.json # datos crudos, corrida 2
docs/mediciones/perf/k6-run3.json # datos crudos, corrida 3
scripts/perf-analysis.py         # script con seed=42, IC 95%
docs/mediciones/perf/REPORT.md   # media, DT, IC 95%, p50/p90/p95/p99
```

En la matriz `docs/trazabilidad/matriz.csv` se lee, para el requisito no funcional de rendimiento:

```
REQ-NF-03,No funcional,Must,,,ProductoService,GET /api/productos,
ProductoPerfTest,CRUD-ORM,docs/mediciones/perf/REPORT.md,verificado
```

La lista siguiente resume el arco completo de las entregas del PFC para situar visualmente la Tercera Entrega.

Arco completo de entregas del PFC (calendario académico 2026)

- **Entrega 1A** (semana del 4 de junio, v0.3.0): consolidación del corpus de ingeniería de requisitos (SRS conforme a ISO/IEC/IEEE 29148:2018, casos de uso, historias de usuario, requisitos no funcionales), diseño arquitectónico C4 nivel 1 y 2, selección y justificación de la pila, esqueleto ejecutable con Docker [11, 5, 6, 4].
- **Entrega 1B** (semana del 14 de junio, v0.7.0): primer módulo funcional del PFC. Módulo de autenticación stateless con JWT (jjwt 0.12, cookie `HttpOnly + Secure + SameSite=Strict`, RFC 7519 con claims estándar), CRUD completo de al menos una entidad principal con Spring Data JPA sobre PostgreSQL 16, migraciones Flyway 9.x, cache Redis 7 para *blacklist* de JTI revocados, Springdoc OpenAPI 2.x en `/api/docs`, Angular 17+ con *interceptor* JWT, docker-compose con `healthcheck`, mínimo cinco pruebas JUnit 5 + MockMvc verdes [13, 15, 27, 18].
- **Entrega 3** (24 de julio, *esta guía*, v0.9.0-rc): aplicación de observaciones acumuladas, consolidación de la ingeniería de requisitos, reproducibilidad automática, evidencia empírica cuantitativa preliminar, trazabilidad completa y publicabilidad del artefacto (FAIR, DOI Zenodo, CITATION.cff, CRediT).
- **Entrega Final** (semana del 17 de agosto, semana 17, v1.0.0): sistema estable con cobertura JaCoCo^a $\geq 70\%$, k6 y Lighthouse dentro de umbrales, auditoría completa de los seis controles OWASP mínimos, puesta en producción en un ambiente públicamente accesible, documento académico final con estructura IMRaD^b ampliada, paquete de datos con DOI propio, y sistema listo para archivo permanente [18, 17, 22, 1].

^a**JaCoCo** (*Java Code Coverage*): herramienta libre que mide qué porcentaje de líneas, ramas y métodos del código Java son ejercitados por las pruebas automatizadas; permite identificar zonas sin cobertura.

^b**IMRaD** (*Introduction, Methods, Results and Discussion*): estructura estándar de los artículos científicos originales de investigación, universal en las revistas indexadas.

2. Requisitos ineludibles de la Tercera Entrega

Los requisitos que siguen consolidan lo entregado en la Entrega 1B y añaden las capas de ingeniería de requisitos formalizada, evidencia empírica y gestión de artefactos que las revista de alto impacto exigen para aceptar un proyecto como reutilizable y reproducible por un tercero sin comunicación con los autores originales [30, 1, 25, 22]. La lectura del apartado debe hacerse en el orden en que aparece: cada bloque supone que el anterior ya está cerrado.

2.1. Bloque 0 — Aplicación integral de las observaciones de las Entregas 1A y 1B

Antes de abordar los bloques técnicos de la Tercera Entrega, el equipo cierra la deuda de calidad acumulada en las entregas previas. Este bloque no es opcional y su verificación es previa a la calificación de cualquier otro bloque.

0.1 — Bitacora de observaciones y su resolución

- Se archiva en el repositorio el archivo `docs/observaciones/OBSERVACIONES.md` que enumera, con código único (`OBS-01`, `OBS-02`, ...), cada observación que el docente formulo en los informes de retroalimentación de las Entregas 1A y 1B. Para cada observación se declara: fuente (Entrega 1A o 1B), criterio de la rúbrica afectado, texto íntegro de la observación, decisión del equipo, y el o los *commits* en los que la resolución quedo aplicada, con el hash corto.
- Cada mensaje de *commit* que resuelve una observación referencia su código (por ejemplo, `fix(auth): rota clave HS256 y firma con RS256 (OBS-07)`).
- El repositorio conserva la etiqueta `v0.7.0` intacta cómo fotografía de la Entrega 1B y añade la etiqueta intermedia `v0.7.1` sobre el commit que cierra la aplicación de observaciones, previa a los trabajos de esta Tercera Entrega.
- El PDF del informe técnico de la Tercera Entrega incluye, cómo primer capítulo después del resumen, una tabla-resumen de las observaciones cerradas con el porcentaje total resuelto y las razones de las no resueltas si las hubiera.

2.2. Bloque A — Consolidacion del sistema entregado en la Entrega 1B

Se exige que la totalidad del alcance de la Entrega 2 permanezca operativo, con las siguientes correcciones y refuerzos, organizados en dos apartados. El apartado A.1 consolida las propiedades funcionales heredadas de la Entrega 2. El apartado A.2 formaliza la estrategia de acceso a datos que rige desde la Tercera Entrega en adelante.

A.1 — Consolidación funcional heredada de la Entrega 2

- Todos los *endpoints* CRUD de la API REST deben mantenerse funcionando y documentados en la especificación *OpenAPI 3.0*⁶ accesible en `/api/docs` [9, 27].
- Toda la autenticación debe operar bajo *cookie* `HttpOnly` + `Secure`⁷ + `SameSite=Strict`⁸ con JWT firmado, siguiendo el flujo definido en el *RFC 7519*⁹ y con los siete *claims*¹⁰ estándar (`iss`, `sub`, `aud`, `exp`, `nbf`, `iat`, `jti`) [13].
- Todos los errores deben responder con *ProblemDetails* de acuerdo con el *RFC 7807*¹¹, incluyendo los campos `type`, `title`, `status`, `detail` y, cuando aplique, `instance` [15].

⁶**OpenAPI 3.0:** estándar abierto para describir de manera formal la interfaz de una API REST, permitiendo generar documentación interactiva (Swagger UI) y clientes automáticos.

⁷**Secure:** atributo de *cookie* que exige que el navegador sólo la envíe a través de conexiones HTTPS, protegiéndola de interceptaciones en redes no cifradas.

⁸**SameSite=Strict:** atributo de *cookie* que impide que el navegador la envíe en solicitudes generadas por sitios de terceros, mitigando ataques *Cross-Site Request Forgery* (CSRF).

⁹**RFC 7519:** estándar del IETF que define el formato JSON Web Token (JWT) para representar afirmaciones seguras entre partes, con firma criptográfica.

¹⁰**Claim:** par clave-valor dentro de un JWT que representa una afirmación sobre el sujeto (por ejemplo, `sub` identifica al usuario, `exp` declara la fecha de expiración).

¹¹**RFC 7807:** estándar del IETF que define un formato uniforme para reportar errores en APIs HTTP, con los campos `type` (URI del tipo de error), `title` (título corto), `status` (código HTTP), `detail` (explicación) e `instance` (URI del caso concreto).

- El *cache* Redis debe seguir en el mismo *endpoint* de listado seleccionado en la Entrega 2, ahora con *TTL*¹² declarado en configuración externa (no en código) y una *hit ratio*¹³ objetivo medida empíricamente y reportada en *docs/mediciones/*.

A.2 — Estrategia obligatoria de acceso a datos

Desde la Tercera Entrega en adelante, el equipo adopta y evidencia una estrategia híbrida de acceso a datos en la que la responsabilidad se reparte con precisión entre el mapeo objeto-relacional¹⁴ y los procedimientos almacenados¹⁵. El sustento formal del enfoque es la especificación *Jakarta Persistence 2.1*, que estandariza el llamado a procedimientos almacenados desde repositorios ORM mediante las anotaciones `@NamedStoredProcedureQuery` y `@StoredProcedureParameter` y la interfaz `StoredProcedureQuery`, con implementación completa en Spring Data JPA a través de la anotación `@Procedure` sobre métodos de repositorio [7, 26]. Ambos mecanismos son reconocidos por la *OWASP Foundation*¹⁶ como defensas primarias equivalentes contra la inyección SQL, siempre que la parametrización sea estricta [19, 18].

A.2.1 — Alcance de los CRUD elementales sobre entidades JPA

Los cuatro CRUD elementales de cada entidad simple se implementan directamente sobre entidades JPA mapeadas con `@Entity`, expuestas como repositorios que extienden `JpaRepository` o `CrudRepository` de Spring Data. Se consideran CRUD elementales, y por tanto se implementan en el ORM:

- La creación de una fila única en una única tabla (**save** sobre una entidad).
- La lectura de una fila por su clave primaria (**findById**).
- El listado completo o paginado de una única tabla, con filtros triviales sobre atributos escalares directos de la propia entidad (**findAll(Pageable)**, **findByCampo**, **findByCampoAndCampo**).
- La actualización de una fila por su clave primaria cuando la operación consiste en asignar valores directos a atributos escalares de la propia entidad.
- La eliminación lógica de una fila por su clave primaria, expresada como cambio de un atributo booleano de la propia entidad.

Todos estos accesos usan métodos derivados de Spring Data o `@Query` con parámetros nombrado; queda prohibido concatenar entrada de usuario en cualquier fragmento JPQL^a, HQL^b o SQL nativo.

¹²**TTL** (*Time To Live*): duración máxima que una entrada permanece en el *cache* antes de ser eliminada automáticamente, expresada en segundos.

¹³**Hit ratio**: proporción de peticiones que se resuelven directamente desde el *cache* sin consultar la base de datos; mide la eficacia de la estrategia de *caching*.

¹⁴**Mapeo objeto-relacional** (ORM, *Object-Relational Mapping*): técnica que traduce automáticamente objetos del código Java (o del lenguaje de la aplicación) a filas de tablas de una base de datos relacional, y viceversa.

¹⁵**Procedimiento almacenado** (*stored procedure*): bloque de código SQL guardado y compilado en el motor de base de datos, invocable con parámetros; encapsula lógica que se ejecuta directamente en el servidor de datos.

¹⁶**OWASP** (*Open Worldwide Application Security Project*): fundación internacional sin ánimo de lucro dedicada a mejorar la seguridad del software; publica guías autoritativas como el *OWASP Top 10* y los *Cheat Sheets* sobre prácticas seguras.

^a**JPQL** (*Jakarta Persistence Query Language*): lenguaje de consultas orientado a objetos definido por la especificación JPA; opera sobre entidades y sus atributos, no directamente sobre tablas y columnas SQL.

^b**HQL** (*Hibernate Query Language*): dialecto propio de Hibernate anterior a JPQL, extendido con características específicas del *framework*.

A.2.2 — Alcance obligatorio de los procedimientos almacenados

Toda operación contra la base de datos que no sea un CRUD elemental según A.2.1 se implementa cómo procedimiento almacenado o función SQL en el motor de base de datos, versionado en el repositorio y llamado desde la aplicación mediante `@Procedure` sobre un método de repositorio Spring Data, o mediante `@NamedStoredProcedureQuery` sobre una entidad, siguiendo el contrato de la especificación JPA 2.1 [7, 26]. Se consideran operaciones obligatoriamente encapsuladas en procedimientos almacenados o funciones, sin excepcion:

- Consultas que involucran mas de una tabla mediante *joins*, sub-consultas o expresiones *common table expressions*.
- Consultas con cálculos agregados: **COUNT**, **SUM**, **AVG**, **MIN**, **MAX**, agrupaciones con **GROUP BY** y filtros **HAVING**.
- Reportes de cualquier naturaleza, incluyendo los que retornan un conjunto de resultados heterogeneo respecto de las entidades del dominio.
- Actualizaciones masivas sobre multiples filas con criterios de selección no triviales.
- Eliminaciones lógicas o físicas condicionadas por reglas de negocio evaluadas sobre mas de una tabla.
- Movimientos transaccionales que afectan a mas de una tabla en la misma unidad de trabajo y requieren validaciones cruzadas o consistencia atomica en el motor.
- Generacion de codigos secuenciales, identificadores de negocio o folios con reglas propias distintas del autoincremento simple.
- Validaciones cruzadas que dependen del estado actual de varias tablas antes de aceptar una operación de escritura.
- Cualquier consulta que retorne una proyección (*DTO*) que no coincide con la estructura de una entidad, incluidas vista materializadas.

Cada procedimiento almacenado o función se define en un archivo SQL versionado bajo `db/procs/` con nombre `sp_<verbo>_<sustantivo>.sql` o `fn_<verbo>_<sustantivo>.sql`, aplica exclusivamente parámetros nombrado, se abstiene de construir SQL dinámico mediante concatenacion (**EXECUTE IMMEDIATE**, **sp_executesql** o equivalentes), y se documenta en `docs/basedatos/CATALOGO-SP.md` con nombre, propósito, parámetros de entrada y salida, cursores devueltos y tabla o tablas afectadas.

A.2.3 — Prohibiciones explicitas y cuenta de aplicación

- Queda prohibido concatenar entrada de usuario en cualquier query, tanto en el código de la aplicación cómo dentro de los procedimientos almacenados.
- Queda prohibido el uso de SQL dinámico dentro de los procedimientos (**EXECUTE**

IMMEDIATE sobre cadenas construidas con concatenacion).

- Queda prohibido depender de `spring.jpa.hibernate.ddl-auto=update` o equivalentes; el esquema y los procedimientos se aplican unicamente desde `db/schema.sql`, `db/seed.sql` y los archivos de `db/procs/` montados en `/docker-entrypoint-initdb.d/`.
- La cuenta de base de datos con la que la aplicación se conecta al motor tiene privilegios mínimos: derechos de `EXECUTE` sobre los procedimientos y funciones necesarios, y derechos de `SELECT`, `INSERT`, `UPDATE` y `DELETE` restringidos a las tablas del dominio; sin `DBA`, `OWNER` ni *superuser* [19].

A.3 — Consolidación de la ingeniería de requisitos

La Tercera Entrega formaliza y consolida el corpus de ingeniería de requisitos que rige el PFC desde la Entrega 1A. La especificación se actualiza a la versión estable candidata, se valida contra el sistema implementado, y se hace trazable de extremo a extremo hacia el código, las pruebas y la evidencia empírica del bloque C. El marco normativo obligatorio es la especificación internacional *ISO/IEC/IEEE 29148:2018*, que define los procesos de ciclo de vida, las plantillas de *Software Requirements Specification* (SRS) y las características de calidad de un requisito bien formado [11]. El equipo complementa este marco con las reglas del *INCOSE Guide to Writing Requirements v4* para redaccion de enunciados individuales y conjuntos [10], con los principios de la ingeniería de requisitos contemporanea documentados por Pohl y por Wiegers y Beatty [20, 29], y con la plantilla *Volere* de Robertson y Robertson para la organización del documento de requisitos [24].

A.3.1 — SRS conforme a ISO/IEC/IEEE 29148:2018

- El repositorio contiene el archivo `docs/requisitos/SRS.pdf` y su fuente `docs/requisitos/SRS.tex` (o Markdown) siguiendo la plantilla de *Software Requirements Specification* de ISO/IEC/IEEE 29148:2018, con las secciones obligatorias: introduccion (propósito, alcance, definiciones, referencias, resumen), descripción global (perspectiva del producto, funciones, características de usuarios, restricciones, supuestos y dependencias), requisitos específicos (funcionales, no funcionales, de interfaz externa, de rendimiento, de seguridad, de calidad de software).
- Cada requisito individual cumple las nueve características de calidad para *requisitos individuales* definidas en el *INCOSE Guide to Writing Requirements v4* (C1–C9): *Necessary*, *Appropriate*, *Unambiguous*, *Complete*, *Singular*, *Feasible*, *Verifiable*, *Correct* y *Conforming*. Además, el conjunto completo de requisitos del sistema cumple las seis características de calidad para *conjuntos de requisitos* (C10–C15): *Complete*, *Consistent*, *Feasible*, *Comprehensible*, *Able to be validated* y *Correct* [10].
- Cada requisito posee un identificador único persistente (REQ-F-001, REQ-NF-001, ...), rationale, atributos de trazabilidad hacia el origen (*stakeholder*), prioridad según *MoSCoW* (*Must*, *Should*, *Could*, *Won't*), criterio de aceptacion medible y

método de verificación (*inspection, analysis, demonstration, test*).

A.3.2 — Historias de usuario y casos de uso

- Las funcionalidades de la Entrega 1B (autenticación, CRUD de la entidad principal) y las que se cierran hasta la Tercera Entrega se especifican cómo historias de usuario en formato *Connextra* (*As a <rol>, I want <objetivo>, so that <beneficio>*), siguiendo los criterios INVEST de Cohn (*Independent, Negotiable, Valuable, Estimable, Small, Testable*) [6]. Cada historia tiene sus criterios de aceptación redactados como *Given/When/Then* de Gherkin.
- Los flujos críticos del sistema (por ejemplo el flujo de autenticación documentado en la Entrega 1B) se detallan también como casos de uso completos siguiendo la plantilla de Cockburn, con los niveles de precisión 1 a 4 (nombre del actor principal y objetivo, escenario principal de éxito, condiciones de extensión, pasos de manejo de extensión) [5].
- Todo el conjunto se archiva bajo `docs/requisitos/historias/` y `docs/requisitos/casos-de-uso/`, ambos versionados en el repositorio y referenciados desde el SRS.

A.3.3 — Matriz de trazabilidad end-to-end

- Se actualiza `docs/trazabilidad/matriz.csv` para que su fila raíz sea el requisito, no el módulo. Las columnas obligatorias son: `id_requisito`, `tipo` (funcional o no funcional), `prioridad_moscow`, `historia_usuario`, `caso_de_uso`, `modulo_codigo`, `endpoint_api`, `prueba_automatizada`, `tipo_acceso` (CRUD-ORM o SP), `evidencia_empirica`, `estado` (pendiente, implementado, verificado).
- La matriz cubre el 100 % de los requisitos con prioridad *Must* vigentes en el estado `v0.9.0-rc`. Los requisitos *Should* y *Could* pueden aparecer con estado **pendiente** pero deben estar presentes en la matriz.
- El *pipeline*^a de CI ejecuta un *script* de validación (`scripts/validate-traceability.sh`) que rechaza commits en los que se agregue un requisito sin correspondencia en al menos una historia, un caso de uso o una prueba.

^a**Pipeline de CI/CD:** secuencia automática de pasos (compilación, pruebas, análisis, despliegue) que se ejecuta ante cada cambio del repositorio; en este PFC se implementa con GitHub Actions y se declara en `.github/workflows/`.

A.3.4 — Gestión del cambio de requisitos

- El archivo `docs/requisitos/CHANGELOG-REQ.md` lleva el registro de cada modificación a los requisitos desde la Entrega 1A, con fecha, autor, requisito afectado, tipo de cambio (*added, modified, deprecated, removed*) y motivo. El formato sigue la convención *Keep a Changelog* adaptada a requisitos.
- Cualquier modificación sustantiva de un requisito *Must* después del cierre de la

Entrega 1B se documenta cómo ADR^a complementario en `docs/adr/` con la plantilla de Nygard [16, 29].

^a**ADR** (*Architecture Decision Record*): documento corto y versionado que registra una decisión arquitectónica importante, con el contexto que la motivó, las alternativas evaluadas, la decisión adoptada y sus consecuencias; formato propuesto por Michael Nygard en 2011.

2.3. Bloque B — Reproducibilidad automática desde clonación limpia

Este bloque es el más exigente para el equipo y el más discriminante para la nota. Su cumplimiento estricto es lo que permite que un revisor externo emita el badge *Artifacts Evaluated — Reusable* definido por la ACM [1].

B.1 — Arranque en un solo comando y sistema autocontenido

- El repositorio debe contener un `Makefile` (o `justfile`, o `Taskfile.yml`) con al menos los objetivos `make up`, `make down`, `make test`, `make bench`, `make audit` y `make clean`. La sola ejecución de `make up` debe levantar el sistema completo sin intervención humana adicional.
- El archivo `docker-compose.yml` debe pinar cada imagen por `digest sha256` (no solo por `tag`), evitando derivas silenciosas entre reconstrucciones.
- Las variables de entorno se centralizan en `.env.example`, comentadas y con valores por defecto seguros para desarrollo local. El archivo `.env` real queda excluido por `.gitignore`.
- El esquema de la base de datos se aplica exclusivamente desde `db/schema.sql` y `db/seed.sql` montados en `/docker-entrypoint-initdb.d/`. Queda prohibido depender de `spring.jpa.hibernate.ddl-auto=update` o similares.
- Todo estado observable de la base al inicio se crea con datos semilla reproducibles: el archivo `db/seed.sql` contiene el usuario `admin` con hash *BCrypt* válido para la contraseña documentada en el README.

B.2 — Determinismo de las mediciones

- Cualquier *script* de generación de datos, de benchmark o de análisis fija su semilla aleatoria (`-seed`, `np.random.seed(42)`, etc.) y la documenta en el propio archivo.
- Los benchmarks `k6` se ejecutan con la misma configuración (50 VUs, duración 30s, *ramp-up* declarado en `k6/opts.js`) para permitir la comparación entre corridas [17].
- Los reportes Lighthouse se generan con `npx lhci autorun` contra un contenedor recién levantado, con perfil móvil declarado y *throttling* de red `Slow 4G`.
- Cada archivo de resultados incluye en su cabecera la fecha ISO 8601, el *commit hash* corto y las versiones exactas de las herramientas (`k6 --version`, `lighthouse --version`, `java -version`).

2.4. Bloque C — Evidencia empirica cuantitativa

La calidad de un proyecto de ingeniería de software se mide por evidencia, no por afirmación. Este bloque exige que cada propiedad no funcional relevante este respaldada por un archivo crudo de mediciones, un *script* de análisis reproducible y un reporte agregado con estadística descriptiva mínima [31, 28].

C.1 — Rendimiento

Se ejecutan al menos tres corridas independientes de la misma prueba de carga con k6 (50 VUs, 30 s) contra el *endpoint* de listado protegido por la API. Para cada corrida se registran los datos crudos en `docs/mediciones/perf/kNN-run{1,2,3}.json` y se calcula:

- Media, mediana, desviación típica e intervalo de confianza al 95 % del tiempo de respuesta.
- Percentiles p50, p90, p95, p99.
- Tasa de errores HTTP ≥ 500 (debe ser cero).
- *Throughput*¹⁷ en peticiones por segundo (media $\pm$ IC 95 %).

Los umbrales objetivo son: p95 de tiempo de respuesta menor a 200ms con cache caliente y menor a 500ms con cache frio. Estos umbrales están alineados con los objetivos de calidad recomendados por la ISO/IEC 25010 para la característica de *Eficiencia de desempeño* [12].

C.2 — Seguridad

Se realiza una auditoría automática de los seis controles OWASP mínimos, con evidencia crudo en `docs/mediciones/sec/` [18].

- **A01** (control de acceso roto): `curl`¹⁸ de un usuario A solicitando el recurso de B debe devolver 403 *Forbidden*. Guardar la salida completa de `curl --include`.
- **A02** (fallo criptografico): capturar `curl -v https://localhost:8443` mostrando TLSv1.3 y suite AEAD.
- **A03** (inyección): enviar POST con payload ' OR '1'='1 en un campo de busqueda; la respuesta esperada es 422 con *ProblemDetails*.
- **A05** (mala configuración de seguridad): `curl -I` contra la raiz y verificar **Strict-Transport-Security**, **X-Frame-Options: DENY**, **X-Content-Type-Options: nosniff** y *Content-Security-Policy*.
- **A07** (fallo de identificacion y autenticación): enviar seis intentos fallidos consecutivos de login desde la misma IP y verificar respuesta 429 *Too Many Requests* desde el sexto en adelante.

¹⁷**Throughput** (rendimiento): cantidad de peticiones que el sistema procesa por unidad de tiempo, usualmente expresada en peticiones por segundo (RPS); es una métrica clave de capacidad y escalabilidad.

¹⁸**curl**: herramienta de línea de comandos que envía peticiones HTTP arbitrarias y muestra las respuestas del servidor; es el instrumento estándar para verificar manualmente el comportamiento de una API REST.

- **A09** (fallo de registro y monitoreo): capturar el archivo de log donde consten las entradas de login exitoso y fallido con `ip`, `timestamp`, `sub` del JWT.

C.3 — Usabilidad

Se aplica el instrumento *System Usability Scale* de Brooke, con las diez preguntas originales sin modificar la escala de cinco puntos [3]. El protocolo mínimo es:

- Poblacion: mínimo diez participantes externos al equipo del PFC, con perfil declarado (edad, sexo, experiencia web, dispositivo). El instrumento SUS ha demostrado validez con muestras desde cinco participantes, pero diez es el mínimo recomendado para producir estimaciones estables de la media.
- Consentimiento informado firmado (físico o digital) y almacenado en `docs/etica/consentimientos/` con datos personales anonimizados (los formularios firmados quedan fuera del repositorio público y se listan por código P01, P02, ...).
- Tarea comun de *onboarding*: login, alta de un registro, edicion, eliminación lógica, logout.
- Resultados: matriz cruda (`docs/mediciones/sus/sus-raw.csv`) con una fila por participante y una columna por item, y reporte agregado con puntuacion media SUS (0 a 100), desviación típica e intervalo de confianza al 95 %.

C.4 — Cobertura de pruebas

Se genera reporte JaCoCo con la cobertura de *lines*, *branches*, *complexity* sobre el módulo de dominio y de servicios de la API. La cobertura objetivo declarada por el planteamiento del PFC es ≥ 70 % para la Entrega Final; en la Tercera Entrega se exige alcanzar al menos ≥ 60 % y una tendencia creciente entre entregas.

C.5 — Accesibilidad y calidad web

Se ejecuta `npx lhci autorun` contra el frontend Angular servido por el contenedor. Los umbrales mínimos declarados en `lighthousec.js` son:

- *Performance* ≥ 80 .
- *Accessibility* ≥ 90 .
- *Best practices* ≥ 90 .
- *SEO* ≥ 90 .

El archivo JSON de cada auditoría queda archivado en `docs/mediciones/lighthouse/` con nombre `lhci-YYYYMMDD-HHMM.json`.

2.5. Bloque D — Documentacion arquitectonica completa y trazable

La arquitectura documentada en la Entrega 2 se refuerza con los siguientes artefactos.

- Diagramas C4 en niveles 1, 2 y 3, con código fuente en *Structurizr DSL* versionado en `docs/arquitectura/` y exportación PNG generada por el pipeline [4].
- Seis ADRs siguiendo la plantilla de Nygard (*Title, Context, Decision, Status, Consequences*), archivados en `docs/adr/adr-NNN-titulo.md`. Los seis temas obligatorios son: elección de la pila principal, esquema de autenticación, gestor de base de datos, estrategia de cache, estrategia de despliegue y estrategia de acceso a datos (separación entre CRUD elementales sobre entidades ORM y operaciones adicionales encapsuladas en procedimientos almacenados o funciones) [16, 7].
- Tabla de atributos de calidad ISO/IEC 25010, con prioridad, escenario y estrategia por cada uno [12].
- Matriz de trazabilidad *requisitos* → *modulos* → *endpoints* → *pruebas* → *evidencia* en `docs/trazabilidad/matriz.csv` o `.md`.

2.6. Bloque E — Publicabilidad del artefacto y gestión de datos

Este bloque orienta la Tercera Entrega hacia el estado de reutilizabilidad exigido por las guías FAIR y por el badge *Artifacts Available* de la ACM [30, 1]. Su cumplimiento se demuestra con archivos concretos dentro del repositorio.

E.1 — Licencia y metadatos de citación

- `LICENSE` en la raíz con el texto íntegro de una licencia aprobada por la Open Source Initiative (MIT, Apache-2.0 o BSD-3-Clause). La elección se justifica en un ADR.
- `CITATION.cff` en la raíz siguiendo la versión 1.2.0 de la especificación, con los campos obligatorios (`cff-version`, `message`, `title`, `authors`) y los recomendados (`version`, `date-released`, `license`, `repository-code`, `doi`, `keywords`) [8, 25].
- Las contribuciones de cada integrante se declaran en `CONTRIBUTORS.md` usando la taxonomía CRediT (14 roles), asignando explícitamente a cada persona los roles en que participa [14].

E.2 — Identificador persistente y archivo del artefacto

- Se crea el archivo Zenodo del repositorio en el estado `v0.9.0-rc`, obteniendo un DOI persistente. El DOI se documenta en el `README`, en `CITATION.cff` y en el propio archivo del PDF de informe.
- El badge Zenodo (`[![DOI](...)]`) aparece al inicio del `README`, junto al badge de *CI passing* y al badge de licencia.

E.3 — Diccionario de datos

- Para cada archivo crudo de mediciones bajo `docs/mediciones/` se registra en `docs/mediciones/DATA-DICTIONARY.md` el nombre de la variable, su tipo de da-

to, la unidad, el rango esperado y su significado.

- El diccionario se actualiza con *pull request* cada vez que se añade una nueva variable, y su vigencia se verifica automáticamente en el pipeline con un test de esquema (por ejemplo con *jsonschema* o *pandera* si se usa Python).

E.4 — Versionado semántico y *changelog*

- El proyecto adopta *Semantic Versioning 2.0.0* de forma estricta desde la Tercera Entrega. La regla MAJOR.MINOR.PATCH se documenta en `docs/VERSIONING.md` y el tag de esta entrega debe ser exactamente `v0.9.0-rc` [21].
- El archivo `CHANGELOG.md` sigue la convención *Keep a Changelog*, con secciones Added, Changed, Deprecated, Removed, Fixed, Security para cada versión.
- Los mensajes de *commit* siguen *Conventional Commits* (`feat:`, `fix:`, `docs:`, `chore:`, `refactor:`, `test:`, `perf:`) para permitir la generación automática del *changelog*.

2.7. Bloque F — Ética y responsabilidad de los datos

Aunque el proyecto no manipule datos de salud, financieros o personales sensibles, la disciplina de investigación contemporánea exige declarar explícitamente los aspectos éticos del uso de datos y de participantes [28, 31].

- `docs/etica/ETHICS.md` declara: (i) fuentes de datos y su licencia, (ii) tratamiento de datos personales si los hubiera, (iii) mecanismo de consentimiento informado para las pruebas de usabilidad, y (iv) ausencia de datos identificables en el repositorio público.
- `docs/etica/consentimientos/plantilla.md` contiene la plantilla de consentimiento informado utilizada en las pruebas SUS del Bloque C.3. Los consentimientos firmados no se suben al repositorio público.

3. Estructura obligatoria del repositorio para la Tercera Entrega

El árbol de directorio que el revisor debe encontrar al clonar el repositorio en el estado `v0.9.0-rc` es el siguiente. Cualquier desviación se considera evidencia de una entrega incompleta y se penaliza en el criterio de calidad de entrega.

Listing 1: Estructura mínima del repositorio para la Tercera Entrega

```
.
|-- README.md           # badges, arranque, credenciales,
|                       # DOI Zenodo, licencia
|-- LICENSE             # texto integro OSI-approved
|-- CITATION.cff        # metadatos de citacion CFF 1.2.0
|-- CONTRIBUTORS.md     # roles CRediT por integrante
|-- CHANGELOG.md        # Keep-a-Changelog
|-- Makefile            # objetivos up down test bench audit
|-- docker-compose.yml  # con digests sha256 pinados
```



```

|-- .env.example          # variables comentadas
|-- .gitignore
|-- backend/             # codigo Spring Boot 3.2.x (Java 21 LTS)
|-- frontend/            # codigo Angular 17+
|-- db/
| |-- schema.sql
| |-- seed.sql
| '-- procs/              # sp_*.sql y fn_*.sql versionados
|-- database/migrations/  # migraciones Flyway V1__, V2__, R__
|-- docs/
| |-- requisitos/
| | |-- SRS.pdf           # ISO/IEC/IEEE 29148:2018
| | |-- SRS.tex           # fuente LaTeX del SRS
| | |-- historias/        # historias INVEST + Gherkin
| | |-- casos-de-uso/     # plantilla Cockburn niveles 1..4
| | '-- CHANGELOG-REQ.md  # Keep-a-Changelog de requisitos
| |-- observaciones/
| | '-- OBSERVACIONES.md  # bitacora de OBS Entregas 1A y 1B
| |-- adr/                # 6 ADRs plantilla Nygard
| | |-- adr-001-...md
| | |-- ...
| | '-- adr-006-...md
| |-- basedatos/
| | '-- CATALOGO-SP.md    # catalogo de procedimientos SP/fn
| |-- arquitectura/       # Structurizr DSL + PNG C4 L1..L3
| |-- mediciones/
| | |-- perf/             # k6 raw runs 1..3 + reporte
| | |-- sec/              # curl -I, curl -v, capturas OWASP
| | |-- sus/              # SUS raw csv + reporte + IC 95%
| | |-- lighthouse/      # JSON lhci por corrida
| | |-- jacoco/           # report.xml + report.html
| | '-- DATA-DICTIONARY.md # variables + tipos + unidades
| |-- trazabilidad/
| | '-- matriz.csv        # req -> historia -> caso uso ->
| |                      #      modulo -> endpoint -> prueba ->
| |                      #      tipo_acceso -> evidencia -> estado
| |-- etica/
| | |-- ETHICS.md
| | '-- consentimientos/plantilla.md
| |-- VERSIONING.md
| '-- informe-entrega-3.pdf # informe tecnico 20-30 paginas
|-- k6/                   # scripts + opts.js
|-- lighthousec.js
|-- scripts/              # analisis con seed fija
| |-- validate-traceability.sh
| '-- audit-sql-dynamic.sh
'-- .github/workflows/     # CI que ejecuta test bench audit

```

4. Entregables consolidados de la Tercera Entrega

El equipo entrega, unicamente a traves del repositorio Git público y del formulario institucional del SGA, los siguientes artefactos.

1. Repositorio Git público con *tag* exactamente igual a **v0.9.0-rc**, cuyo *commit hash* corto se declara en la portada del PDF de informe técnico.
2. DOI persistente asignado por Zenodo al tag **v0.9.0-rc**. El DOI se declara en tres lugares (README, CITATION.cff, portada del PDF) y debe resolver a la misma versión.
3. PDF de informe técnico de 20 a 30 páginas, ubicado en **docs/informe-entrega-3.pdf**, con las secciones: portada, resumen ejecutivo, estado del sistema, arquitectura actual (C4 L1–L3), matriz de trazabilidad, protocolo experimental, resultados por bloque (rendimiento, seguridad, usabilidad, cobertura, accesibilidad), amenazas a la validez, declaración de contribuciones (CRediT), declaración ética, referencias.
4. Coleccion Postman actualizada (**docs/postman/coleccion.json**) con al menos 20 peticiones cubriendo casos de éxito, error de validación (422), no autorizado (401), prohibido (403) y no encontrado (404).
5. Video corto (2–3 minutos, alojado enlazado desde el README) donde el equipo demuestra **make up**, ejecución de **make bench** y **make audit**, y muestra los reportes generados. No sustituye a la evidencia archivada.

5. Rubrica de evaluacion de la Tercera Entrega

La rúbrica pondera once criterios agrupados en tres ejes: eje de continuidad y requisitos (criterios C0 y C0R, 22 %), eje de ingeniería (criterios C1–C6, 53 %) y eje de publicabilidad del artefacto (criterios C7–C10, 25 %). Cada criterio se evalúa en cinco niveles: *Excelente* = 100 %, *Satisfactorio* = 75 %, *En desarrollo* = 50 %, *Insuficiente* = 25 %, *Ausente* = 0 %. La escala de cuatro niveles activos mas Ausente ya fue empleada por el docente en la rúbrica GA-PE-U2 y se mantiene aquí por consistencia interna del curso.

5.1. Formula de calificacion

$$\text{Nota}_{100} = \sum_{i=1}^{11} (\text{peso}_i \times \text{nivel}_i) \quad \text{Nota}_{10} = \frac{\text{Nota}_{100}}{10}$$

5.2. Rubrica detallada

Criterio	Peso	Excelente (100 %)	Satisfactorio (75 %)	En desarrollo (50 %)	Insuficiente (25 %)
C0. Aplicacion de observaciones de Entregas 1A y 1B (Bloque 0)	10 %	docs/observaciones presente con el 100 % de las observaciones cerradas, cada una con código, texto íntegro, decisión del equipo y commit hash de resolución; commits referencian el código (OBS-NN); tag v0.7.1 colocado sobre el commit de cierre de observaciones.	docs/observaciones/OBSERVACIONES.md presente con el 100 % de las observaciones cerradas con trazabilidad correcta; tag v0.7.1 presente.	Entre 60 % y 80 % de observaciones cerradas o trazabilidad parcial.	Menos del 60 % de observaciones cerradas o ausencia del archivo.
C0R. Consolidacion de ingenieria de requisitos (Bloque A.3)	12 %	SRS conforme a ISO/IEC/IEEE 29148:2018 versionado en docs/requisitos/100 % de los requisitos <i>Must</i> con identificador, rationale, prioridad MoSCoW, criterio de aceptacion medible y método de verificación; historias de usuario en formato Connextra con INVEST y Gherkin; casos de uso Cockburn niveles 1–4; reglas INCOSE v4 aplicadas a la redaccion; matriz de trazabilidad end-to-end operativa con validación en CI.	SRS presente con estructura ISO/IEC/IEEE 29148:2018 y ≥ 80 % de requisitos con atributos completos; historias y casos de uso presentes con calidad aceptable; matriz cubre ≥ 80 % de <i>Must</i> .	SRS presente pero con estructura parcial; historias o casos de uso incompletos; matriz de trazabilidad con vacios.	Sin SRS formal o requisitos redactados sin cumplir las quince características INCOSE.

Criterio	Peso	Excelente (100 %)	Satisfactorio (75 %)	En desarrollo (50 %)	Insuficiente (25 %)
C1. Consolidación funcional y estrategia de acceso a datos (Bloque A.1 y A.2)	6 %	Todos los <i>endpoints</i> de la Entrega 1B operativos, JWT en cookie HttpOnly+Secure+SameSite, ProblemDetails RFC 7807 en el 100 % de errores, cache Redis operativo con TTL externo, y separación CRUD-elementales-en-ORM frente a operaciones-adicionales-en-SP evidenciada con al menos un procedimiento almacenado invocado desde un repositorio Spring Data y catalogado en <code>docs/basedatos/CATALOGO-SP.md</code> .	Igual pero con una discrepancia menor documentada o con catálogo de SP parcial.	Al menos un <i>endpoint</i> caído, ProblemDetails en menos del 80 % de errores, o ausencia del catálogo de SPs pese a existir operaciones adicionales a los CRUD elementales.	Regresiones funcionales significativas respecto a la Entrega 1B o ausencia total de procedimientos almacenados para las operaciones adicionales.
C2. Reproducibilidad automática (Bloque B.1)	10 %	<code>make up</code> arranca todo desde clonación limpia sin intervención; imágenes pinadas por digest sha256; <code>.env.example</code> completo; seed y schema montados en <code>docker-entrypoint</code>	Arranque en un solo comando con una variable de entorno adicional documentada.	Requiere pasos manuales no documentados o falla ocasional.	No arranca desde clonación limpia siguiendo el README.
C3. Determinismo de mediciones (Bloque B.2)	7 %	Todos los scripts con semilla fija y versiones de herramientas capturadas en cada archivo de resultados; 3 corridas independientes por benchmark.	Semillas fijas en scripts principales; menos de 3 corridas o versiones parcialmente registradas.	Semillas presentes en algunos scripts; corridas únicas.	Sin semillas fijas ni control de versiones de herramientas.

Criterio	Peso	Excelente (100 %)	Satisfactorio (75 %)	En desarrollo (50 %)	Insuficiente (25 %)
C4. Evidencia empírica cuantitativa (Bloque C)	10 %	Los cinco sub-bloques (perf, sec, sus, cobertura, lighthouse) con archivos crudos, scripts de análisis y reportes con media, IC 95 % y percentiles; SUS con ≥ 10 participantes.	Cuatro de los cinco sub-bloques completos; el faltante justificado en el informe.	Tres de los cinco sub-bloques; SUS con menos de 10 participantes.	Menos de tres sub-bloques con evidencia; SUS ausente.
C5. Documentación arquitectónica y trazabilidad (Bloque D)	8 %	C4 L1–L3 versionado en Structurizr DSL; cinco ADRs con plantilla Nygard completa; tabla ISO 25010; matriz de trazabilidad completa <i>req</i> → <i>código</i> → <i>prueba</i> → <i>evidencia</i> .	Cuatro ADRs completos; matriz cubre ≥ 80 % de requisitos.	Tres ADRs; matriz parcial.	Menos de tres ADRs y matriz ausente o superficial.
C6. Auditoría de seguridad OWASP (Bloque C.2 con énfasis)	12 %	Los seis controles OWASP evidenciados con <code>curl</code> verificable, respuestas HTTP correctas (403/422/429), cabeceras HSTS, CSP, X-Frame-Options, X-Content-Type-Options presentes; logs con timestamp, IP y sub.	Cinco controles evidenciados; una cabecera menor faltante.	Cuatro controles evidenciados.	Tres o menos controles evidenciados.
C7. Licencia, citación y contribución (Bloque E.1)	5 %	LICENSE OSI-approved presente; CITATION.cff v1.2.0 válido; CONTRIBUTORS.md con roles CRediT explícitos por integrante.	Licencia y CITATION.cff presentes; roles CRediT parciales.	Licencia presente; CITATION.cff ausente o inválido.	Sin licencia declarada.

Criterio	Peso	Excelente (100 %)	Satisfactorio (75 %)	En desarrollo (50 %)	Insuficiente (25 %)
C8. Archivo permanente e identificador (Bloque E.2)	5 %	DOI Zenodo asignado al tag <code>v0.9.0-rc</code> , declarado en tres lugares y resolviendo a la misma versión.	DOI asignado y declarado en dos lugares.	DOI asignado pero no declarado en el repositorio.	Sin DOI persistente.
C9. Datos, diccionario y versionado (Bloques E.3, E.4)	7 %	Datos crudos en <code>docs/mediciones/</code> ; <code>DATA-DICTIONARY.md</code> completo; SemVer 2.0.0 estricto; <code>CHANGELOG.md</code> con Keep-a-Changelog; commits siguen Conventional Commits.	Diccionario cubre ≥ 80 % de variables; changelog presente.	Diccionario parcial; changelog irregular.	Sin diccionario o sin changelog.
C10. Ética y calidad de la entrega global (Bloque F + integridad global)	8 %	<code>ETHICS.md</code> y plantilla de consentimiento presentes; PDF de informe con las 10 secciones exigidas; matriz de amenazas a la validez presente; historial Git con ≥ 30 commits granulares y autoría correcta de todos los integrantes.	Documentación ética presente; informe con una sección faltante justificada.	<code>ETHICS.md</code> mínimo; informe con dos secciones faltante.	Sin declaración ética ni informe estructurado.
Total ponderado		100 % $\text{Nota}_{100} = \sum(\text{peso}_i \times \text{nivel}_i)$ $\text{Nota}_{10} = \text{Nota}_{100}/10$			

5.3. Reglas transversales de calificación

Reglas transversales que aplican a todos los criterios

1. **El repositorio Git es la fuente de verdad.** La evidencia verificada directamente en el repositorio se distingue siempre de la inferida de un PDF o de capturas.
2. Los criterios dependientes de código (C1–C6) se califican *Ausente* (0 %) cuando no existe repositorio público accesible o cuando el tag `v0.9.0-rc` no existe.
3. Cualquier commit posterior al corte declarado en la primera página del informe se ignora para efectos de calificación, pero no se penaliza (queda cómo evidencia de

trabajo posterior a la entrega).

4. Si el `make up` falla desde una clonación limpia siguiendo estrictamente el README, el criterio C2 se califica automáticamente *Insuficiente* (25 %) o menos, independientemente del resto.
5. Si el DOI declarado no resuelve al tag `v0.9.0-rc`, el criterio C8 se califica *Ausente* (0 %).
6. Si el `CITATION.cff` no valida contra el esquema `cff-version: 1.2.0` publicado en Zenodo (validado con `cffconvert`), el criterio C7 se limita a *En desarrollo* (50 %) como máximo [8].
7. Cualquier evidencia de concatenación de entrada de usuario en JPQL, HQL, SQL nativo o dentro de un procedimiento almacenado, y cualquier uso de SQL dinámico construido por concatenación (`EXECUTE IMMEDIATE`, `sp_executesql` o equivalentes), califica automáticamente el criterio C1 como *Insuficiente* (25 %) y el criterio C6 como *Insuficiente* (25 %) o menos, independientemente del resto de la evidencia [19, 18].
8. Todos los archivos crudos de mediciones deben conservarse tal cual los produjo la herramienta original; su edición manual invalida la evidencia.

6. Consideraciones para la Entrega Final (semana 16)

La Tercera Entrega deja al equipo en el estado `v0.9.0-rc`, es decir, un candidato a versión estable. La transición a `v1.0.0` en la Entrega Final consistirá en absorber las observaciones de la evaluación de esta entrega, elevar la cobertura JaCoCo desde el mínimo del 60 % hasta el objetivo del 70 % o superior, completar las mediciones que la Tercera Entrega haya dejado incompletas, ampliar la muestra SUS si aplica y consolidar el archivo permanente. En consecuencia, ninguna decisión arquitectónica mayor debe posponerse a la Entrega Final: si algo requiere reescribir un ADR o refactorizar un módulo, debe ocurrir dentro de la Tercera Entrega.

Referencias

- [1] Association for Computing Machinery. Artifact review and badging — version 1.1, August 2020. Version 1.1 (August 24, 2020).
- [2] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 4th edition, 2021.
- [3] John Brooke. SUS: A ‘quick and dirty’ usability scale. In Patrick W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, editors, *Usability Evaluation in Industry*, pages 189–194. Taylor and Francis, London, 1996.
- [4] Simon Brown. *The C4 Model for Visualising Software Architecture*. Leanpub, 2023.
- [5] Alistair Cockburn. *Writing Effective Use Cases*. Agile Software Development Series. Addison-Wesley Professional, Boston, MA, 2001.

- [6] Mike Cohn. *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional, Boston, MA, 2004.
- [7] Linda DeMichiel. JSR 338: Java(TM) persistence API, version 2.1 — final release. Technical Report JSR-338, Oracle America, Inc. — Java Community Process, May 2013. Final Release, 16 May 2013.
- [8] Stephan Druskat, Jurriaan H. Spaaks, Neil Chue Hong, Robert Haines, James Baker, Spencer Bliven, Egon Willighagen, David Pérez-Suárez, and Alexander Kononov. Citation file format, 2021.
- [9] Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [10] International Council on Systems Engineering. INCOSE guide to writing requirements. Technical Report INCOSE-TP-2010-006-04, International Council on Systems Engineering (INCOSE) — Requirements Working Group, July 2023.
- [11] International Organization for Standardization, International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 29148:2018, systems and software engineering — life cycle processes — requirements engineering. Technical Report ISO/IEC/IEEE 29148:2018, ISO/IEC/IEEE, November 2018.
- [12] ISO/IEC. ISO/IEC 25010:2011 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE). International Organization for Standardization, 2011.
- [13] Michael Jones, John Bradley, and Nat Sakimura. JSON web token (JWT). RFC 7519, Internet Engineering Task Force (IETF), 2015.
- [14] National Information Standards Organization (NISO). ANSI/NISO Z39.104-2022, CRediT, contributor roles taxonomy. Technical report, National Information Standards Organization, Baltimore, MD, February 2022.
- [15] Mark Nottingham and Erik Wilde. Problem details for HTTP APIs. RFC 7807, Internet Engineering Task Force (IETF), 2016.
- [16] Michael Nygard. Documenting architecture decisions. Cognitect Blog, November 2011.
- [17] Jose Manuel Ortega. *Mastering Python for Networking and Security: Leverage the scripts and libraries of Python version 3.7 and beyond to overcome networking and security issues*. Packt Publishing, 2nd edition, 2022.
- [18] OWASP Foundation. OWASP top 10:2021 — the ten most critical web application security risks, 2021.
- [19] OWASP Foundation. SQL injection prevention cheat sheet. OWASP Cheat Sheet Series, 2024. Documento vivo mantenido por la comunidad.

- [20] Klaus Pohl. *Requirements Engineering: Fundamentals, Principles, and Techniques*. Springer, Heidelberg, 2010.
- [21] Tom Preston-Werner. Semantic versioning 2.0.0, 2013.
- [22] Paul Ralph, Nauman bin Ali, Sebastian Baltes, Domenico Bianculli, Jessica Diaz, Yvonne Dittrich, Neil Ernst, Michael Felderer, Robert Feldt, Antonio Filieri, Breno Bernard Nicolau de França, Carlo Alberto Furia, Gregory Gay, Nicolas Gold, Daniel Graziotin, Pinjia He, Rashina Hoda, Natalia Juristo, Barbara Kitchenham, Valentina Lenarduzzi, Jorge Martínez, Jorge Melegati, Daniel Mendez, Tim Menzies, Jefferson Moller, Dietmar Pfahl, Romain Robbes, Daniel Russo, Nytyi Saarimäki, Federica Sarro, Davide Taibi, Janet Siegmund, Diomidis Spinellis, Mirosław Staron, Klaas Stol, Margaret-Anne Storey, Damian Tamburri, Marco Torchiano, Christoph Treude, Burak Turhan, and Sira Vegas. Empirical standards for software engineering research, 2021.
- [23] Mark Richards and Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, 2020.
- [24] Suzanne Robertson and James Robertson. *Mastering the Requirements Process: Getting Requirements Right*. Addison-Wesley Professional, Boston, MA, 3 edition, 2012.
- [25] Arfon M. Smith, Daniel S. Katz, Kyle E. Niemeyer, and FORCE11 Software Citation Working Group. Software citation principles. *PeerJ Computer Science*, 2:e86, 2016.
- [26] Spring Data Team. Spring data JPA reference documentation — stored procedures. VMware — Broadcom (Spring by VMware), 2025.
- [27] Craig Walls. *Spring in Action*. Manning Publications, 6th edition, 2022.
- [28] Hironori Washizaki, editor. *SWEBOK Guide v4.0: Software Engineering Body of Knowledge*. IEEE Computer Society, 2024.
- [29] Karl E. Wiegers and Joy Beatty. *Software Requirements*. Microsoft Press, Redmond, WA, 3 edition, 2013.
- [30] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, Niklas Blomberg, Jan-Willem Boiten, Luiz Bonino da Silva Santos, Philip E. Bourne, et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016.
- [31] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg, 2012.